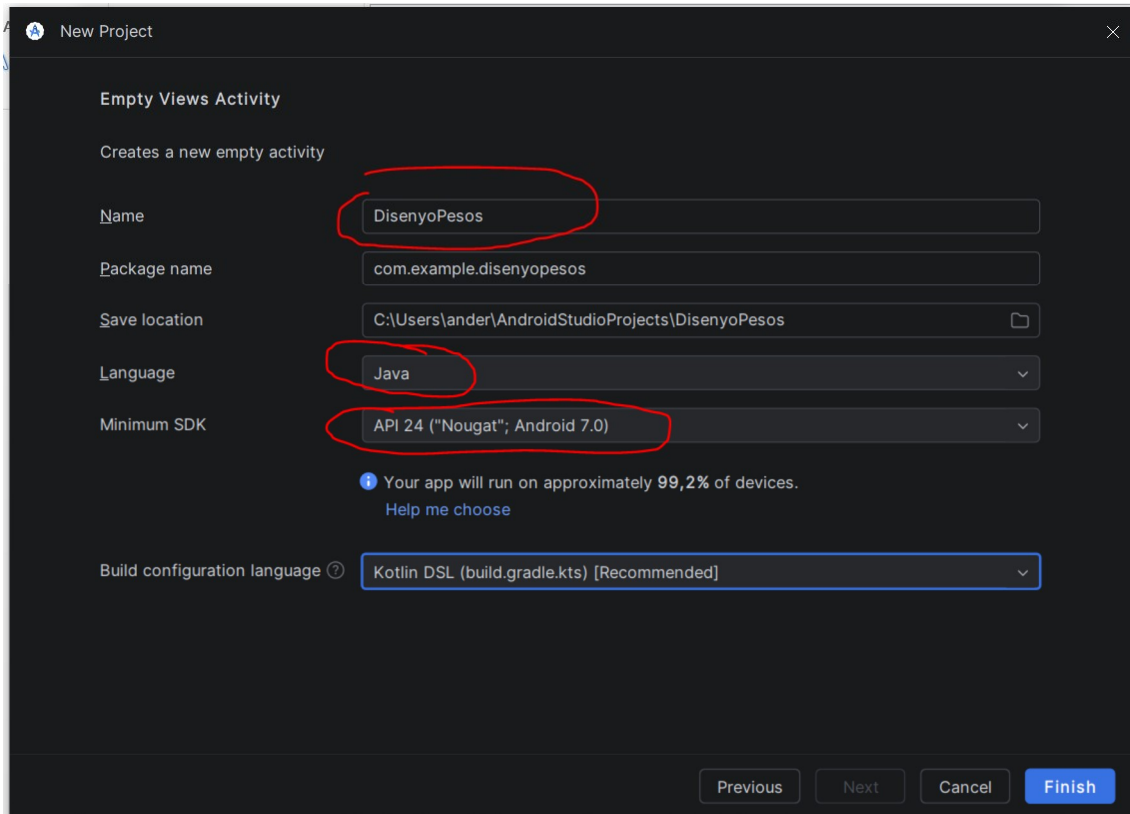


Diseño basado en pesos

En primer lugar se comenzará por crear un proyecto vacío. En el menú se seleccionará Android 7.0 y una 'Empty views activity' para comenzar con una actividad vacía. Será importante seleccionar Java como lenguaje:



Una vez cargada la interfaz será importante comprobar en la parte superior izquierda de Android Studio que la vista seleccionada es Android.

En la parte izquierda podrá verse la estructura del proyecto, en la que cabe destacar los siguientes archivos:

- **manifests/AndroidManifest.xml**: archivo que contiene los permisos de la App, así como declaración de actividades y el tema seleccionado.
- **java/.../MainActivity.java**: es el código Java que maneja la actividad principal. Irá asociado a un archivo XML que contiene el diseño.
- **res/layout/activity_main.xml**: es el archivo XML que contiene el diseño de la actividad principal.
- **res/values/strings.xml**: es el archivo XML que contiene las etiquetas de texto de la app.
- **res/values/dimens.xml**: contiene las dimensiones de la app. Habrá que crearlo, puesto que no aparece por defecto.
- **res/values/colors.xml**: contiene los colores de la app.
- **res/values/themes/themes.xml**: contiene los estilos de la app.
- **res/mipmap/**: es el directorio donde se guarda el icono de la app.

- **res/drawable/**: es el directorio donde se guardan imágenes en versiones de Android anteriores a la 12. Además, sirve para guardar las áreas o selectores (cambios de estado).
- **gradle**: contiene la configuración de la exportación de la app y contiene los enlaces a las librerías.

Una actividad es un concepto de Android que representa una ‘ventana’, algo así como un elemento con el que el usuario puede interactuar.

Si abrimos el `activity_main.xml` nos encontraremos el siguiente código, del que hay que sustituir la parte en del `ConstraintLayout` por `LinearLayout`:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:id="@+id/main"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity"
android:orientation="vertical">

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!" />

</LinearLayout>
```

De esta forma se pasa la actividad a un `LinearLayout`. Un layout es un método para situar los distintos componentes gráficos dentro de la actividad.

Para definir los colores de los bloques de nuestra ventana habrá que abrir el archivo `res/values/colors.xml` y **agregar 4 nodos**. Para elegir el color se podrá clicar en el rectángulo de la izquierda, que abrirá un color picker:

```
<color name="rojo">#de070a</color>
<color name="gris">#a4a4a4</color>
<color name="verde">#35f238</color>
<color name="blanco">#ffffff</color>
```

Con el `LinearLayout` los elementos se colocan en horizontal o vertical según el atributo `orientation`. Para repartir el espacio entre ellos se usa el atributo `weight`: cuanto mayor es el peso de un elemento, más espacio ocupa, de forma proporcional. La clave es que la dimensión que se reparte debe valer `0dp`: en un layout vertical se pone `layout_height="0dp"` y en uno horizontal `layout_width="0dp"`. Así, con pesos 2, 7 y 1, los bloques ocupan respectivamente 2/10, 7/10 y 1/10 del alto disponible.

En siguiente lugar se eliminará el `TextView` que viene por defecto con la plantilla y se dividirá la app en 3 partes, cada una con su correspondiente `LinearLayout`:

- Cabecera.

- Cuerpo
- Pie

Para ello se crearán otras 3 etiquetas LinearLayout dentro de la anterior. A cada una se le fija width="match_parent", height="0dp" y un weight (la proporción que ocupará), más un color:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:id="@+id/main"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity"
android:orientation="vertical">

    <!-- CABECERA -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="2"
        android:background="@color/rojo" />

    <!-- CUERPO -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="7"
        android:background="@color/gris" />

    <!-- PIE -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:background="@color/verde" />

</LinearLayout>
```

Se agregan en res/values/strings.xml las siguientes etiquetas que contendrán los textos para el título:

```
<string name="titulo1">Centro de Estudios Almi
Ikastetxea</string>
<string name="curso">DAM 2</string>
```

Para los tamaños se crea en res->values ->new->xml->values resource file un archivo dims.xml y se ha codificado lo siguiente:

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<resources>

    <dimen name="tamanoTitulo">20sp</dimen>

    <dimen name="tamanoSubtitulo">10sp</dimen>

    <dimen name="espacioTitulo">7dp</dimen>

</resources>
```

Para dar estilos a los títulos se crearán 2 estilos, que heredarán el uno del otro. Editando el archivo `res/values/themes/themes.xml` se **agregará** lo siguiente:

```
<style name="cursoTitulo">
    <item name="android:gravity">center</item>
    <item name="android:textStyle">bold</item>
    <item name="android:textColor">@color/blanco</item>
    <item name="android:textSize">@dimen/tamanoTitulo</item>
    <item name="android:padding">@dimen/espacioTitulo</item>
</style>

<style name="cursoTitulo.subtitulo">
    <item
name="android:textSize">@dimen/tamanoSubtitulo</item>
</style>
```

En cuanto a las unidades es importante destacar que se suele utilizar `sp` para los textos y `dp` para las distancias (ambas son unidades que se adaptan al tamaño de pantalla). El estilo `subtitulo` es una herencia del de `título`, que simplemente sobrecarga la propiedad `TextSize`. Se monta el encabezado en `res/layout/activity_main.xml`. Se crean dentro dos `TextView` con los estilos correspondientes. En:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:id="@+id/main"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity"
android:orientation="vertical">

    <!-- CABECERA -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="2"
        android:background="@color/rojo"
        android:orientation="vertical"
        android:gravity="center">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
```

```

        android:text="@string/titulo1"
        style="@style/cursoTitulo" />

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/curso"
            style="@style/cursoTitulo.subtitulo" />
    </LinearLayout>

    <!-- CUERPO -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="7"
        android:background="@color/gris" />

    <!-- PIE -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:background="@color/verde" />
</LinearLayout>

```

Una vez dado el contenido al encabezado el siguiente paso será crear 6 botones en el cuerpo, cada uno de los cuales tendrá una etiqueta de texto para indicar la funcionalidad del botón. Cada uno de los botones tendrá una serie de imágenes, por lo que habrá que introducir dichas imágenes en el proyecto. Para ello habrá que poner Android Studio en vista proyecto y copiar todos los archivos de imagen en `app/src/main/res/drawable`.

Para formar la interfaz la estrategia a utilizar será crear una línea que contendrá un conjunto de botón y etiqueta a la izquierda y lo mismo en la derecha, de forma que posteriormente se copiará y pegará todo ello para crear las 2 líneas de botones de debajo. En el `Layout gris` habrá que agregar la orientación vertical para que vaya apilando los botones. Dentro del mismo `Layout` habrá que agregar otro `LinearLayout` que representará la línea de botones, que tendrá `width` y `height` como `match_parent` y `0dp`, `weight 1` y `orientation horizontal` para que sitúe los botones uno al lado del otro.

Una vez creada la línea habrá que crear dos `LinearLayout` para contener el botón y la etiqueta de texto, por lo que a diferencia del anterior se les dará `orientation vertical` y `gravity center` para que los textos aparezcan centrados. Una vez creado este `LinearLayout` se creará dentro del mismo una etiqueta `ImageButton` a la que se le asignará la imagen `al` a través del parámetro `background` y un `TextView` al que se asignará un `weight de 1` res/ por cuestiones de diseño. Para crear el par de botón y texto de la derecha solo habrá que copiar y pegar este `LinearLayout` debajo:

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"

```

```
xmlns:tools="http://schemas.android.com/tools"
android:id="@+id/main"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity"
android:orientation="vertical">

<!-- CABECERA -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="2"
    android:background="@color/rojo"
    android:orientation="vertical"
    android:gravity="center">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/titulo1"
        style="@style/cursorTitulo" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/cursor"
        style="@style/cursorTitulo.subtitulo" />
</LinearLayout>

<!-- CUERPO -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="7"
    android:background="@color/gris"
    android:orientation="vertical">

    <!-- FILA 1 -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:orientation="horizontal">

        <!-- Celda izquierda -->
        <LinearLayout
            android:layout_width="0dp"
            android:layout_height="match_parent"
            android:layout_weight="1"
            android:orientation="vertical"
            android:gravity="center">

            <ImageButton
                android:layout_width="match_parent"
                android:layout_height="0dp"
                android:layout_weight="3"
                android:background="@drawable/a1" />

            <TextView
                android:layout_width="match_parent"
```

```

        android:layout_height="0dp"
        android:layout_weight="1"
        android:text="TOAST"
        android:gravity="center" />
    </LinearLayout>

    <!-- Celda derecha -->
    <LinearLayout
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="1"
        android:orientation="vertical"
        android:gravity="center">

        <ImageButton
            android:layout_width="match_parent"
            android:layout_height="0dp"
            android:layout_weight="3"
            android:background="@drawable/a1" />

        <TextView
            android:layout_width="match_parent"
            android:layout_height="0dp"
            android:layout_weight="1"
            android:text="ASYNCTASK"
            android:gravity="center" />
    </LinearLayout>
</LinearLayout>

<!-- PIE -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:background="@color/verde" />
</LinearLayout>

```

Para crear las siguientes 2 líneas de botones simplemente habrá que copiar y pegar el código de la fila. Finalmente queda la interfaz de la siguiente forma:



Una vez creada la parte principal de la interfaz se personalizarán los textos de cada botón. Para ellos se agregan los siguientes `res/values/strings.xml`:

```
<string name="toast">TOAST</string>
<string name="asynctask">ASYNCTASK</string>
<string name="adapter">ADAPTER</string>
<string name="frame">FRAME BY FRAME</string>
<string name="tween">TWEEN</string>
<string name="property">PROPERTY</string>
```

Se fija en los `TextView` esos strings:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity"
    android:orientation="vertical">

    <!-- CABECERA -->

    <LinearLayout
        android:layout_width="match_parent"
```

```

        android:layout_height="0dp"
        android:layout_weight="2"
        android:background="@color/rojo"
        android:orientation="vertical"
        android:gravity="center">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/titulo1"
            style="@style/cursorTitulo" />

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/cursor"
            style="@style/cursorTitulo.subtitulo" />
    </LinearLayout>

    <!-- CUERPO -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="7"
        android:background="@color/gris"
        android:orientation="vertical">

        <!-- FILA 1 -->
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="0dp"
            android:layout_weight="1"

```

```

        android:orientation="horizontal">

        <LinearLayout
            android:layout_width="0dp"
            android:layout_height="match_parent"
            android:layout_weight="1"
            android:orientation="vertical"
            android:gravity="center">
            <ImageButton
                android:layout_width="match_parent"
                android:layout_height="0dp"
                android:layout_weight="3"
                android:background="@drawable/a1" />
            <TextView
                android:layout_width="match_parent"
                android:layout_height="0dp"
                android:layout_weight="1"
                android:text="@string/toast"
                android:gravity="center" />
            </LinearLayout>
        <LinearLayout
            android:layout_width="0dp"
            android:layout_height="match_parent"
            android:layout_weight="1"
            android:orientation="vertical"
            android:gravity="center">
            <ImageButton
                android:layout_width="match_parent"
                android:layout_height="0dp"
                android:layout_weight="3"
                android:background="@drawable/a1" />

```

```

        <TextView
            android:layout_width="match_parent"
            android:layout_height="0dp"
            android:layout_weight="1"
            android:text="@string/asyncTask"
            android:gravity="center" />

    </LinearLayout>
</LinearLayout>

<!-- FILA 2 -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:orientation="horizontal">

    <LinearLayout
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="1"
        android:orientation="vertical"
        android:gravity="center">

        <ImageButton
            android:layout_width="match_parent"
            android:layout_height="0dp"
            android:layout_weight="3"
            android:background="@drawable/a1" />

        <TextView
            android:layout_width="match_parent"
            android:layout_height="0dp"
            android:layout_weight="1"
            android:text="@string/adapter"

```

```

        android:gravity="center" />
    </LinearLayout>

    <LinearLayout
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="1"
        android:orientation="vertical"
        android:gravity="center">
        <ImageButton
            android:layout_width="match_parent"
            android:layout_height="0dp"
            android:layout_weight="3"
            android:background="@drawable/al" />
        <TextView
            android:layout_width="match_parent"
            android:layout_height="0dp"
            android:layout_weight="1"
            android:text="@string/frame"
            android:gravity="center" />
    </LinearLayout>
</LinearLayout>

```

```

<!-- FILA 3 -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:orientation="horizontal">

    <LinearLayout
        android:layout_width="0dp"

```

```

        android:layout_height="match_parent"
        android:layout_weight="1"
        android:orientation="vertical"
        android:gravity="center">
        <ImageButton
            android:layout_width="match_parent"
            android:layout_height="0dp"
            android:layout_weight="3"
            android:background="@drawable/a1" />
        <TextView
            android:layout_width="match_parent"
            android:layout_height="0dp"
            android:layout_weight="1"
            android:text="@string/tween"
            android:gravity="center" />
    </LinearLayout>

    <LinearLayout
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="1"
        android:orientation="vertical"
        android:gravity="center">
        <ImageButton
            android:layout_width="match_parent"
            android:layout_height="0dp"
            android:layout_weight="3"
            android:background="@drawable/a1" />
        <TextView
            android:layout_width="match_parent"
            android:layout_height="0dp"
            android:layout_weight="1"

```

```

        android:text="@string/property"
        android:gravity="center" />
    </LinearLayout>
</LinearLayout>
</LinearLayout>

<!-- PIE -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:background="@color/verde" />
</LinearLayout>

```

Se agrega para el pie de página una línea en res/values/strings.xml:

```
<string name="pie">www.almi.eus</string>
```

Después habrá que ir modificando la propiedad Text de cada uno de los TextView. También habrá que agregar un TextView al pie para poner la firma:

```

<!--*****PIE*****-->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:background="@color/verde"
    android:gravity="center">
    <TextView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:gravity="center"
        android:text="@string/pie"/>
    </LinearLayout>

```

Una vez aprendido el funcionamiento básico de la configuración se optará por trabajar con otros aspectos, como son los fondos o los cambios de estado de los botones.

Comenzando con el fondo de la App, se tratará de eliminar los colores anteriormente fijados y crear un degradado para el fondo de la app. Para definir el fondo habrá que hacer clic derecho en `res/drawable` en el árbol del proyecto de Android Studio y hacer clic en `new-> drawable resource file`, al que llamaremos `fondo.xml`. Dentro de dicho fichero habrá que crear el siguiente código eliminando su contenido:

```
<?xml version="1.0" encoding="utf-8"?>
<shape
xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <gradient
        android:startColor="#00bfff"
        android:endColor="#000bff"
        android:centerColor="#e0f8f7"
        android:angle="270" />
</shape>
```

Una vez realizado esto al `LinearLayout` general de la app habrá que fijarle el fondo del gradiente recién creado y eliminar los fondos de cada uno de los `LinearLayout`:

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity"
    android:orientation="vertical"
    android:background="@drawable/fondo">
```

Será necesario también **quitar** las líneas que dan color a los contenedores:

```
...
android:background="@color/rojo"

...

android:background="@color/gris"

...

android:background="@color/verde"
```

A continuación se creará un nuevo `drawable` de nombre `cabecera.xml` con el siguiente código para crear un marco para la cabecera con el siguiente código:

```
<?xml version="1.0" encoding="utf-8"?>
<shape
  xmlns:android="http://schemas.android.com/apk/res/android"
  android:shape="rectangle">

  <solid android:color="@android:color/transparent" />

  <padding
    android:left="8dp"
    android:right="8dp"
    android:top="2dp"
    android:bottom="2dp" />

  <corners android:radius="48dp" />

  <stroke
    android:color="@color/blanco"
    android:width="2dp" />
</shape>
```

Con la etiqueta `solid` se generará el fondo de la forma. Es importante fijar que el fondo de la forma sea transparente, ya que en algunas versiones de Android el color por defecto es el blanco y no el transparente, por lo que podría tapar el contenido del encabezado. El `padding` controlará el espacio entre el marco y el contenido, mientras que los `corners` serán las cuatro esquinas de la forma. En cuanto al `stroke` es el marco de la forma que se está generando.

Por último habrá que poner en el `LinearLayout` de la cabecera que se use el marco recién generado:

```
<!-- CABECERA -->
<LinearLayout
  android:layout_width="match_parent"
  android:layout_height="0dp"
  android:layout_weight="2"
  android:orientation="vertical"
  android:gravity="center"
  android:background="@drawable/cabecera">
```

Por último se creará un selector, también en `drawable` de nombre `botón_pulsado.xml`, para que los estados de los botones 'pulsado' y 'no pulsado' se distingan bien:

```
<?xml version="1.0" encoding="utf-8"?>
<selector
  xmlns:android="http://schemas.android.com/apk/res/android">

  <!-- PULSADO -->
  <item
    android:drawable="@drawable/a2"
    android:state_pressed="true" />
```

```
<!-- NO PULSADO -->
<item
    android:drawable="@drawable/a1"
    android:state_pressed="false" />
</selector>
```

Y cambiar el fondo de los botones:

```
<ImageButton
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="3"
    android:background="@drawable/boton_pulsado" />
```

El final será poner la app a pantalla completa. Para ello se modifica MainActivity.java eliminando todo lo que está tras el setContentView:

```
package com.example.disenyopesos;

import android.os.Bundle;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_main);
    }
}
```

El resultado final de la app será el siguiente:

